Week 9: Arrays and functions

ID1063 Introduction to Programming in C
Lecture 17

Announcements

- Revised dates for the Midsem Lab exam will be announced by this weekend.
  - Potential dates: Oct 3rd / Oct 10th .
  - Syllabus: Until and including Week 9.
  - No lecture on Fri (non-instructional day); will inform if changed.

This week

- Arrays as arguments to functions
- Two-dimensional arrays
- Topics deferred earlier: long int and string.h

Element Frequencies

- Problem from earlier : count frequency of each number occurrence in an array a[1…n]

- Repeated job: given an element x, find the number of occurrences of x in the array

- It is logical write a function for this, but what should the function take as arguments?
  - Call that function for target values a[0], a[1], a[2], and so on

int countOccurrences (int arr[], x)
array

Functions involving arrays

Functions with arrays as parameters require slightly careful specification. There are two methods:

Method 1:

```
// To compute sum of elements in an array

int arraySum(int arr[], int count)
```

values: the array elements

IMPORTANT: Array size is **not** specified in []

count: included to indicate length of the array

NOTE: count need not be actual length

Array arguments

- Give the array name without [] in the argument

```
// function declaration
int arraySum(int arr[], int count) {
…
}

//Function call:
arraySum(temperatures, max);
```

Alternative

- **Method 2:**

```
// To compute sum of elements in an array
int arraySum(int count, int arr[count])
```

- The logical array length is count. It is specified **before** we use arr.
  - The compiler needs to know what count is
- Your code should still be careful to not access out-of-bounds location (beyond count): the compiler will **not** check it.
- Both methods are perfectly fine. Some non gcc-compilers prefer method 1. In C++, only method 1 is typically allowed.
  - Method 1 additionally follows the logical method of func(data, size)

Capacity v/s Logical size

Capacity

- Maximum number of elements the array can hold
- Fixed when the array is declared
- Example: int data[100] has capacity 100

func (arr, size)
↓
at most the array capacity

Logical size

- Number of elements currently relevant
- Stored in a separate variable
- Every array function needs the relevant logical size

- We have already encountered example of this with strings.

Back to counting occurrences

```
int countOccurrences(int arr[], int n, int target);

int main(void) {
    int data[6] = {3, 1, 3, 4, 3, 2};
    int count = countOccurrences(data, 6, 3);
    printf("%d\n", count);
    return 0;
}
```

The call uses the array name data. **No brackets appear in the call.**

countOccurrences

```
int countOccurrences(int values[], int n, int target) {
    int count = 0;

    for (int i = 0; i < n; i = i + 1) {
        if (values[i] == target) {
            count = count + 1;
        }
    }

    return count;
}
```

Similar functions

```
int arraySum(int values[], int n);
int arrayMaximum(int values[], int n);
int countOccurrences(int values[], int n, int target);
```

Each helper receives the array and the number of elements to examine.

arraySum

```
int arraySum(int values[], int n) {
    int total = 0;

    for (int i = 0; i < n; i = i + 1) {
        total = total + values[i];
    }

    return total;
}

int main(void) {
    int data[4] = {3, 1, 4, 2};
    int n = 4;
    int total = arraySum(data, n);
    printf("%d\n", total);
    return 0;
}
```

total is local to arraySum. The returned value enters main.

Recall: Pass by value for scalars

```
void addOne(int number) {
    number = number + 1;
}

int main(void) {
    int count = 4;
    addOne(count);
    printf("%d\n", count);   /* prints 4 */
    return 0;
}
```

What is printed?

Ans: 4

Pass by value for arrays?

```
void addOneToEach(int values[], int n) {
    for (int i = 0; i < n; i = i + 1) {
        values[i] = values[i] + 1;
    }
}

int main(void) {
    int data[3] = {4, 7, 9};
    addOneToEach(data, 3);
    printf("%d %d %d\n", data[0], data[1], data[2]);
    return 0;
}
```

What is printed here?

5 8 10

What C passes to a function

Scalar argument

- The function receives a copied scalar value
- Changing the parameter changes only the copy
- The caller's scalar remains unchanged

Array argument

- The function can access the caller's array elements
- Changing an element is visible after the call

Reason for this difference is because C internally thinks of array variables as *pointers*, more on this next week.

Functions with strings

A string is a char array ending in '\0', so an explicit size is usually not needed.

Function header for finding length of a string:

```
int manualLength(char text[])
```

Manual string length

```
int manualLength(char text[]) {
    int length = 0;
    while (text[length] != '\0') {
        length = length + 1;
    }
    return length;
}
```

Two-dimensional arrays

A table of values

	0	1	2
0	12	7	5
1	9	4	11

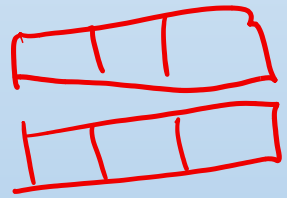
int matrix [2][3];

matrix[0][1] is 7

Declaration and initialization

```
int matrix[2][3] = {
 {12, 7, 5},
 { 9, 4, 11}
};

printf("%d\n", matrix[0][1]);   /* prints 7 */
```



IMPORTANT: **Unlike 1D arrays, for initializing 2D arrays, the sizes have to be specified.**

Valid row indices: 0 and 1. Valid column indices: 0, 1, and 2.

Reading a 2D array

```
for (int row = 0; row < numrows; row = row + 1) {
    for (int column = 0; column < numcolumns;
         column = column + 1) {
        scanf("%d", &matrix[row][column]);
    }
}
```

Outer loop: Rows, Inner loop: Columns

Printing a 2D array

```
for (int row = 0; row < rows; row = row + 1) {
    for (int column = 0; column < columns;
         column = column + 1) {
        printf("%4d", matrix[row][column]);
    }
    printf("\n");
}
```

The newline belongs after the inner loop.

Row sums

```
for (int row = 0; row < numrows; row = row + 1) {
    int rowSum = 0;

    for (int column = 0; column < numcolumns;
         column = column + 1) {
        rowSum = rowSum + matrix[row][column];
    }

    printf("Row %d sum: %d\n", row, rowSum);
}
```

The program prints the sum of elements in each row.

Reset rowSum to zero before processing each row.

Exercise: Print the sum of entries in each column.